TCS NQT · ADVANCED CODING · ACTUAL QUESTIONS

Advanced Coding Actual Questions Collection

Seven coding problems recalled from candidates' screen photos of the TCS Campus Hiring (FY23) Advanced Coding section, cleaned up, categorised by topic and solved with verified approaches.

7	7	11	5
actual problems	topic categories	worked examples	verified solutions

ONLINE LEARNING MOCK TEST SERIES · ATC DEVELOPERS

About this collection

These problems come from photos of the TCS Campus Hiring (FY23) **Advanced Section-2**, where the candidate gets two coding problems (the **Advanced Coding-1** and **Advanced Coding-2** tabs) and must press **Final Submit** for the program to be evaluated. The photos were taken of a screen, so some parts were cut off or hard to read.

Please read:

- The problem statements below are **typed out from the photos**. Wording follows the original as closely as possible, with obvious spelling slips fixed. Where a part of the statement, an example or the constraints could not be seen in any photo, this is said clearly under "Notes about the photos".
- The titles and the topic and difficulty labels are ours, to help you study. They are not part of the real exam.
- The photos carried candidate watermarks, which are not reproduced here.
- Solutions are marked **verified** only when we ran them on the examples from the statement and, where possible, compared them with a brute-force search. A solution built on a guess about a missing part of the statement is marked as a **possible reading**.

All problems at a glance

#	Problem	Topic	Level	Tab	Exam date shown
1	**String Conversion with the Smallest Letter**	Strings	Hard	Advanced Coding-1	19 Aug 2022, Slot 1
2	**Jersey and Board: Beats of the Drum**	Math and Number Theory	Medium	Advanced Coding-2	(FY23 series), date not visible
3	**Tina and the Perfect Square Cloth**	Geometry and Coordinates	Medium	Advanced Coding-2	(FY23 series), date not visible
4	**Alice and the Verbal Kho-Kho T-shirts**	Arrays and Simulation	Easy to Medium	Advanced Coding-1	(FY23 series), date not visible
5	**The Golden House: Rooms with Exactly K Coins**	Subarray Sum and Prefix Sums	Medium	Advanced Coding-1	20 Aug 2022, Slot 1
6	**Reddington Restructures His Stocks**	Greedy and Sorting	Hard	Advanced Coding-1	21 Aug 2022, Slot 1
7	**Radha's Sweet Packing Contest**	Hashing, Sets and Simulation	Medium	Advanced Coding-1	(FY23 series), date not visible

By topic

Strings	Q1 String Conversion with the Smallest Letter
Math and Number Theory	Q2 Jersey and Board: Beats of the Drum
Geometry and Coordinates	Q3 Tina and the Perfect Square Cloth
Arrays and Simulation	Q4 Alice and the Verbal Kho-Kho T-shirts

Subarray Sum and Prefix Sums	Q5 The Golden House: Rooms with Exactly K Coins
Greedy and Sorting	Q6 Reddington Restructures His Stocks
Hashing, Sets and Simulation	Q7 Radha's Sweet Packing Contest

By difficulty (our estimate)

Easy to Medium	Q4
Medium	Q2, Q3, Q5, Q7
Hard	Q1, Q6

Suggested order for practice

Start with **Q5** (prefix sums), **Q2** (permutation cycles and LCM) and **Q4** (counting). Then do **Q3** (coordinates with a hash set), **Q7** (sets and simulation). Finish with the two hardest: **Q1** (string conversion, a greedy argument) and **Q6** (greedy with sorting).

Common input and output rules seen in these problems

- The system does not allow hard-coded input; your program is tested on inputs supplied by the system.
- Print only what is asked. Extra messages in the output make the test case fail.
- Read N first, then the next lines as described; use fast input for N up to 10^5.

Q1 String Conversion with the Smallest Letter

TCS ADVANCED CODING · ADVANCED CODING-1

Strings | Hard | Advanced Coding-1 | TCS Campus Hiring FY23, 19 Aug 2022, Slot 1

Problem statement

Aayush is working on a strange algorithm where he wants to convert a string A to B. Both the strings are of equal length N.

Below are the rules which can be performed to convert the string:

- String A and B are of equal length N.
- Both of them are in lower case letters.
- Choose a subset X from the string A, between the index 1 and N.
- Let s be the letter which alphabetically comes before all other letters in this subset. Let s be called the "smallest element" in the subset.
- Replace all the elements of the subset with this letter s.

Find the minimum number of moves which is required to perform this conversion. If it is not possible to convert the string from A to B, then return -1.

Let us try to understand it with an example. Suppose there are 2 strings, A = abcab and B = aabab.

Operation 1: Now we have chosen a subset S, let us say we have taken index 2, 3, 5 from A. Then the subset S becomes [bcb]. Next, we have to choose the smallest element, which is b here, in b and c. Now we will place all the respective elements back to their respective index. This will update the original string as [aabab]. [see note below]

This is exactly same as String B. Hence it is possible to convert string A to B, with 2 operations. So, the answer is 2.

Input format

The photos show the first line as N followed by the two strings A and B, one per line (see Example 1). The "input format for testing" section is not visible in the photos.

Output format

A single integer: the minimum number of moves, or -1 if the conversion is not possible.

Constraints

Not visible in the photos.

Example 1

Input

```
2
de
cd
```

Output

```
-1
```

Explanation: In the above example we can clearly see that there is an alphabet in A which is completely [not convertible] (the rest of the explanation is cut off in the photo). Each move can only replace letters by a smaller (or equal) letter, so "d" can never become "c" when "c" does not exist in A.

Notes about the photos

- In the photo the 5-line "Operation 1" walkthrough shows the result [aabab] after placing the elements back; the exact intermediate string is partly cut. Our own check: choosing positions 2, 3, 5 of abcab gives a b b a b, and a second move on positions 1, 2 gives aabab, so 2 moves is correct.
- Example 1 input is shown as "2 / de / cd" (N = 2, A = de, B = cd) with output -1.

How to think about it

- Letters can only go down. If any A[i] < B[i], print -1.
- Every position with A[i] > B[i] needs a move whose smallest letter is B[i]. That move needs some position currently holding that letter B[i]. Because letters only go down, such a letter must already exist somewhere in A. If it does not exist, print -1.
- Moves for different target letters can be done in decreasing order of letter, and every position that needs the same target letter can be fixed together in one move. So the answer is the number of **distinct target letters** that are needed.

Complexity: O(N). **Check:** Checked against both examples and against a brute-force search over all moves on 400 random small strings (no mismatch).

Solution (Python)

```
def solve(A, B):
    if any(a < b for a, b in zip(A, B)):
        return -1
    need = {b for a, b in zip(A, B) if a > b}
    if any(t not in set(A) for t in need):
        return -1
    return len(need)

# solve("abcab", "aabab") -> 2 ;  solve("de", "cd") -> -1
```

Q2 TCS ADVANCED CODING · ADVANCED CODING-2
Jersey and Board: Beats of the Drum

Math and Number Theory | Medium | Advanced Coding-2 | TCS Campus Hiring (FY23 series), date not visible

Problem statement

Jack is a sports teacher at St. Patrick's School. He makes games not only to make the student fit, but also smart. So, he lined up all the N number of students in his class. At each position he has fixed a board with the integer number printed on it. Each of the numbers are unique and are in exactly the range of N. Let us say there are 10 students, then the boards will be printed with numbers from 1 to 10 in a random order given by the sequence A[].

As a rule, all students wear a jersey with their numbers printed on it. So, if there are N students, each will have a unique jersey number, just like a football team.

Now, in the beginning, all the students will stand as per the increasing order of their jersey numbers, from left to right.

The only difference will be their respective board number which is placed at their respective location. The board location is fixed and cannot be changed. We can consider the arrangement as below. Suppose there are 5 students, and the board is placed in the order of [2 3 1 5 4]:

```
Board -- 2, 3, 1, 5, 4
Student's Jersey -- 1, 2, 3, 4, 5
```

Now the game begins. After every beat of the drum, each student will have to move to that location (index), where his board is pointing to. In the above case student with jersey #1 is standing with board #2, so now he will have to move to location #2. Similarly, all the other students will do. So after first beat of the drum, the alignment will be:

```
Board -- 2, 3, 1, 5, 4
Student's Jersey -- 3, 1, 2, 5, 4
```

Given N and the order of board of the respective positions, find the number of beats required to bring back the students to their original position. So, for the above case the answer is 6.

Input format

First line: N. Second line: the board alignment B[] (N integers). (Example 1 is shown as "3" and "{1, 2, 3}".)

Output format

A single integer: the number of beats.

Constraints

Not visible in the photos.

Example 1

Input

```
3
1 2 3
```

Output

```
1
```

Explanation: All the students will be standing as below, with the board positions: Board 1, 2, 3 and Student's Jersey 1, 2, 3. After the first beat of the drum: Jersey #1 will move to index 1, jersey #2 will move to index 2, jersey #3 will move to index 3. (Already in original positions after 1 beat.)

Notes about the photos

- The explanation text of Example 1 is cut at the bottom of the photo; the answer 1 is clear from the output line.

How to think about it

- The board array is a permutation. One beat moves the student at index i to index B[i], so the students move along the cycles of this permutation.
- After t beats every student is back home only when t is a multiple of every cycle length. So the answer is the **LCM of all cycle lengths**. For [2 3 1 5 4] the cycles are (1 2 3) and (4 5): LCM(3, 2) = 6.

Complexity: O(N) time, O(N) space. **Check:** Checked against both numbers in the statement and against direct simulation of the beats on 300 random permutations (no mismatch). If N is large, the answer can be big; use big integers or the modulus given in the real constraints.

Solution (Python)

```
from math import gcd

def beats(B):
    n = len(B); seen = [False] * n; ans = 1
    for i in range(n):
        if not seen[i]:
            j, length = i, 0
            while not seen[j]:
                seen[j] = True; j = B[j] - 1; length += 1
            ans = ans * length // gcd(ans, length)
    return ans

# beats([2,3,1,5,4]) -> 6 ;  beats([1,2,3]) -> 1
```

Q3 TCS ADVANCED CODING · ADVANCED CODING-2
Tina and the Perfect Square Cloth

Geometry and Coordinates | Medium | Advanced Coding-2

TCS Campus Hiring (FY23 series), date not visible

Problem statement

Tina was given a piece of silk cloth. There are already a few points or coordinates mentioned on it. She is creating something which needs an exact square shape cloth.

Help Tina to find out how many minimum points she can add, to cut the perfect square from the given cloth. There are already a few points marked on the cloth. Tina has to include the most number of points from the given points and should try to include the minimum number of points or coordinates from her side.

Find the minimum number of coordinates, to achieve the perfect square. Also, try to get as bigger cloth as possible.

Let us try to understand it with an example. Let's say there is 3 points given, means N = 3. These points are: (0,0), (2,2), (3,3). We can have 2 additional co-ordinates: a. (2,0) and (0,2) OR b. (3,0) and (0,3). Tina doesn't want to lose a good deal here, so better she would go with (3,0) and (0,3). Hence 2 additional coordinates were required to cut a perfect square from that piece of cloth. Hence the answer is 2.

Input format

First input: the integer N. Next N lines: integers X and Y separated by a SPACE.

Output format

An integer as per the above logic (check the output in Example 1 and Example 2). Additional messages in output will cause the failure of test cases.

Constraints

1 <= N <= 1000
-999 <= X, Y <= 999

Example 1

Input

```
5
0 0
100 100
200 200
100 0
0 100
```

Output

```
0
```

Explanation: In the above scenario, we can already make a square from the given coordinates: P1 (0,0), P2 (100,0), P3 (0,100), P4 (100,100). Hence no needs for any additional coordinates here. (The photo prints "So the answer is No" at the end of this explanation; the output line says 0.)

Example 2

Input

Output

```
3
0 0
2 2
3 3
```

```
2
```

Explanation: In the above scenario we can have 2 additional co-ordinates: a. (2,0) and (0,2) OR b. (3,0) and (0,3). Tina don't want to lose a good deal here, so better she would go with (3,0) and (0,3). Hence 2 additional co-ordinates were required to cut a perfect square from that piece of cloth. Hence the answer is 2.

Notes about the photos

- The statement never says the square must have its sides parallel to the axes, but both examples use axis-parallel squares. The approach below assumes that.

How to think about it

- A square has 4 corners. The answer is 4 minus the largest number of given points that can be corners of the same square (at least 1, because any single point can be a corner).
- Take every pair of points. If $|dx| = |dy|$ they can be opposite corners; if $dx = 0$ or $dy = 0$ they can be neighbouring corners (two possible squares). For each candidate square count how many of the other two corners are in a hash set. $O(N^2)$ pairs is fine for N up to 1000.
- The "bigger cloth" remark only breaks ties between squares that need the same number of added points, so it does not change the printed count.

Complexity: $O(N^2)$. **Check:** Checked against both examples. The axis-parallel assumption comes from the examples and is not stated in the visible text.

Solution (Python)

```python
def tina(pts):
    S = set(pts); best = 1
    for (x1, y1) in pts:
        for (x2, y2) in pts:
            if (x1, y1) == (x2, y2): continue
            dx, dy = x2 - x1, y2 - y1
            if abs(dx) == abs(dy):                      # opposite corners
                best = max(best, 2 + ((x1, y2) in S) + ((x2, y1) in S))
            if dy == 0:                                 # side along x
                for s in (dx, -dx):
                    best = max(best, 2 + ((x1, y1 + s) in S) + ((x2, y2 + s) in S))
            if dx == 0:                                 # side along y
                for s in (dy, -dy):
                    best = max(best, 2 + ((x1 + s, y1) in S) + ((x2 + s, y2) in S))
    return 4 - best

# Example 1 -> 0 ; Example 2 -> 2
```

Q4 TCS ADVANCED CODING · ADVANCED CODING-1
Alice and the Verbal Kho-Kho T-shirts

Arrays and Simulation | Easy to Medium | Advanced Coding-1

TCS Campus Hiring (FY23 series), date not visible

Problem statement

Alice and her friends are playing a game of verbal Kho-Kho. Alice is acting as a mediator, and the rest of the N friends are seated on N chairs, one each.

Alice starts by providing a paper with a single digit number to the friend present at number 1. Let's denote friends by F, where F will be of size N. F[1]...F[N] represents friends seated respectively.

After receiving the paper with a digit, F[1] will enact and try to tell F[2] without speaking. Similarly, F[2] will communicate to the next person i.e. F[3]. This continues until the last person F[N] understands the digit.

Finally, the last person will write the digit on a separate paper and will give it to Alice.

Alice will compare both the papers. If the digits are similar then, Alice will give a T-shirt to each friend. However, if the digits do not match then Alice will ask digits from each friend, and she will offer the T-shirts to only the ones who understood the digits correctly.

Given N number of friends and digit array D, denoting the digit understood by each friend F, find out how many of Alice's friends have not enacted well OR did not understand the enactment by the previous friend correctly.

Input format

First line: contains a positive integer denoting N.
Next line: contains N elements of the array, space separated, denoting array D.

Output format

Output the single integer denoting how many have not enacted well OR have not understood the enactment.

Constraints

1 <= N < 10000
0 <= D[i] <= 9

Example 1

Input

```
3
4 4 4
```

Output

```
0
```

Explanation: All of them have understood the digits correctly.

Example 2

Input

```
5
1 2 3 2 2
```

Output

```
4
```

Explanation: 1st, 2nd, 3rd and 4th friends could not enact OR understand the enactment.

Notes about the photos

- The statement does not say clearly what the first digit Alice gave is; the photos give no further hint.
- The wording of the Example 2 explanation (1st, 2nd, 3rd and 4th friends) is as printed in the photo. Two natural readings of the rule give 4 for this example, so please confirm the exact rule with the original statement before relying on it.

How to think about it

- **Possible reading (not verified against hidden tests):** if the first and the last digit are equal, nobody is penalised: print 0. Otherwise count the friends whose digit differs from the first friend's digit. This gives 0 for Example 1 and 4 for Example 2.

Complexity: O(N). **Check:** Both examples are consistent with the reading above; no further verification was possible because the full statement is ambiguous.

Q5 The Golden House: Rooms with Exactly K Coins

TCS ADVANCED CODING · ADVANCED CODING-1

Subarray Sum and Prefix Sums | Medium | Advanced Coding-1 | TCS Campus Hiring FY23, 20 Aug 2022, Slot 1

Problem statement

This vacation you went to visit the golden house. There are N rooms in this golden house and its owner needs someone to take care of the management of this house. As you have been unemployed for a long time, you are interested in this job. Owner of this house wanted an intelligent manager for this role, so he created one puzzle within that golden house. The person who will be able to solve that puzzle will be the manager of the golden house.

Owner of the house kept some numbers of golden coins in each room. You have to choose two rooms, one from where you will enter and other one from where you will exit. From any room either you can exit, or you can move to the next room. While visiting any room you will collect all the gold coins, and if you enter any room then you can't skip collecting gold coins from that room, you have to take those coins. Owner wants to have exactly K coins, when you exit the room, he guarantees that there will be at least one possible solution for this puzzle.

Given number of rooms (N), and number of gold coins in all N rooms. You have to find room numbers from where you will start and from where you will exit. If there are more than one solution possible, then you have to provide a solution with a smaller starting room number. You can assume that room numbers will start from 1 and end at N.

Hint: Find a continuous subsequence whose sum will be exactly equal to K.

Input format

First line: N and K (two integers). Second line: N integers, the number of gold coins in each room. (The "input format for testing" section is not visible in the photos.)

Output format

Two integers: the starting room number and the exit room number.

Constraints

Not visible in the photos.

Example 1

Input

```
10 15
5 3 7 14 18 1 18 4 8 3
```

Output

```
1 3
```

Explanation: There are ten rooms in the house. You want to have the total sum of gold coins in continuous sequence of room to be 15. There are two solutions from this i.e. [1 3] and [8 10]. As 1 is smaller than 8 so final solution be [1 3].

Example 2

Input

Output

```
1 5
```

```
5 25
3 4 1 8 9
```

Explanation: There are five rooms and you want the total sum of gold coins ... (the explanation is cut off in the photo; the whole array sums to 25, so the answer is rooms 1 to 5).

Notes about the photos

- In the photo the input of Example 1 is shown as "10 15 → N = 10, K = 15" and then the coins on the next line.
- The last sentence of the Example 2 explanation is cut off at the bottom of the photo.

How to think about it

- Find a contiguous block of rooms whose coins add up to exactly K and whose starting room is the smallest.
- If all coin counts are positive, a sliding window (two pointers) works in O(N). If zeros or negative numbers can appear, use prefix sums with a hash map that remembers the first index of every prefix value; scanning from the left still gives the smallest start.

Complexity: O(N). **Check:** Checked against both examples in the statement and against a brute-force search.

Solution (Python)

```python
def rooms(K, a):
    first = {0: 0}            # prefix sum -> smallest prefix length
    s = 0; best = None
    for i, x in enumerate(a, 1):
        s += x
        if s - K in first:
            start = first[s - K] + 1
            if best is None or start < best[0]:
                best = (start, i)
        first.setdefault(s, i)
    return best

# rooms(15, [5,3,7,14,18,1,18,4,8,3]) -> (1, 3)
# rooms(25, [3,4,1,8,9])              -> (1, 5)
```

Q6 TCS ADVANCED CODING · ADVANCED CODING-1
Reddington Restructures His Stocks

Greedy and Sorting | Hard | Advanced Coding-1 | TCS Campus Hiring FY23, 21 Aug 2022, Slot 1

Problem statement

Reddington has invested his money in stock of N companies, but lately he realized that he has invested money in a very random manner. He wants to restructure the money he invested.

He decided an integer K which he will use to modify the amount invested either by putting K more amount for a particular company or withdrawing the same K amount.

He will withdraw from one company (by selling stock) and will invest the same amount K to another company (by buying stock).

For more than one transaction (either buy or sell) of stock in the same company have some charges per month, such that he will not do more than one transaction of a particular company means either he will buy or sell stock of amount K for a company or he will not do any transaction for that company.

But he can do as many transactions as he wants provided that in one company he will do only one transaction.

As you are his best friend so he reached out to you for help.

Reddington wants to keep the difference of smallest and largest invested amount minimal, can you please help him to get this minimum difference?

NOTE: Reddington can sell stock of K from a particular company only if the invested amount value for that particular company is higher than K.

Hint: To keep the difference minimum, if any transaction is needed then he needs to sell stock of amount K from the company with the highest invested amount and invest the same in the company with the smallest invested amount.

Input format

The candidate has to write the code to accept 2 inputs. First input: it will contain two integers N (no. of companies) and K (integer which Reddington will use for transaction) separated by space. Second input: it will contain N integers, each separated by space, which will be the amount invested in N companies.

Output format

The output will contain a single integer which will be the minimum possible difference of amount invested with largest and smallest amount. Additional messages in output will cause the failure of test cases.

Constraints

$1 <= N <= 10^5$

$1 <= K <= 10^9$

$1 <= C0, C1, C2, ..., Cn <= 10^9$

Example 1

Input

Output

```
2 100
100 1000
```

```
700
```

Explanation: Two companies C1 and C2 having invested amounts as 100 and 1000. From company C2 Reddington will sell stock of 100 (K) unit and buy stock of company C1. So, now the invested amount will be 200 and 900, and now Reddington can't do more transactions, so the minimum difference will be 900 - 200 = 700.

Example 2

Input

```
5 100
100 200 300 400 500
```

Output

```
200
```

Explanation: Five companies C1, C2, C3, C4, C5 having invested amounts as 100, 200, 300, 400, 500. Reddington will sell stock of C5 of 100 units and put it in C1. So, the invested amount will be 200, 200, 300, 400, 400. Now Reddington will sell stock of C4 and put in C2, then the invested amount will be 200, 300, 300, 300, 400. Reddington can't do more transactions now, so the minimum difference will be 400 - 200 = 200.

Notes about the photos

- In the photo of Example 1 the explanation says "From company C2 ... buy stock of company C2"; it is a typo for C1 (the smaller amount). The numbers 200 and 900 make it clear.

How to think about it

- Sort the amounts. Doing t transactions means selling K from the t largest companies and buying K for the t smallest ones (each company is used at most once, and a sale needs amount > K).
- For each t from 0 to N/2, the new minimum and maximum come from only a few values: the untouched middle, the sold ones (largest minus K) and the bought ones (smallest plus K). Take the best difference over all t.

Complexity: O(N log N). **Check:** Checked against both examples and against an exhaustive search over every possible buy/sell choice on 500 random small cases (no mismatch).

Solution (Python)

```python
def stocks(a, K):
    a = sorted(a); n = len(a)
    best = a[-1] - a[0]
    for t in range(1, n // 2 + 1):
        if a[n - t] <= K:          # the t-th largest cannot sell K
            break
        mx = max(a[n - t - 1] if n - t - 1 >= t else -1, a[-1] - K, a[t - 1] + K)
        mn = min(a[t] if t <= n - t - 1 else 10**18, a[n - t] - K, a[0] + K)
        best = min(best, mx - mn)
    return best

# stocks([100,1000], 100) -> 700 ; stocks([100,200,300,400,500], 100) -> 200
```

Q7 TCS ADVANCED CODING · ADVANCED CODING-1
Radha's Sweet Packing Contest

Hashing, Sets and Simulation | Medium | Advanced Coding-1

TCS Campus Hiring (FY23 series), date not visible

Problem statement

Radha is attending her sister's wedding, and there are a lot of things to do including packing and food and decoration. Radha took up the responsibility for packing the sweets, as no one was interested in doing such a tedious work. She got an idea, where she announced that we will pick a winner from a packing contest, and those who win this game will be gifted a surprise gift. Best part is more than 1 person can be winner in this game.

Radha, by default is the first player. So, the game goes like this:

- All the sweet boxes are represented with numbers, in order to differentiate with each other. Let us say that a box numbered with '1' will contain a different sweet compared to the one which is numbered '2'.
- Each player will be given different/same combinations of sweets boxes in different/same quantities.
- Each player has to pack all the respective boxes given to them, in one final package.
- As default point, each person will be given the number of points equal to the number of boxes they pack. Let us say they have 6 sweet boxes in their queue, they will get 6 points as default.
- If the final package contains 4 different types of sweets, then they will get additional 1 point ... *[the rest of this rule is cut between the two photos]*

Clearly player 2 is a winner with more points. So, the answer is 2.

Input format

First input: accept the value of integer N. Next N lines: elements of sequence Z[] (each row contains Z[0] number of elements, i.e. the first number of the row is the count of boxes of that player).

Output format

The output is an integer / Radha / Tie as per the above logic (check the output in Example 1, Example 2). Additional messages in output will cause the failure of test cases. Tie: if 2 or more than 2 players have the same score. A[i]: index of the player.

Constraints

1 <= N <= 20

1 <= Z[i] <= 100

Example 1

Input

```
2
6 1 2 3 4 5 6
9 3 3 3 4 4 4 5 5 5
```

Output

```
Radha
```

Explanation: Player 1: Radha. Has 6 boxes, so 6 points in the beginning. She has 6 different types of sweets (from 1-6), hence 6 more points to that. Total of 6 + 6 = 12 points.

Player 2: Has 9 boxes, so 9 points in the beginning. She has only 3 different types of sweets (3, 4 and 5), so no points to that. Total of 9 points.
Clearly Radha is a winner with more points.

Notes about the photos

- The photo of the rule is cut after "additional 1 point"; Example 1 shows that 6 different types give 6 more points and 3 types give none, so the bonus is probably 1 point per different type once there are at least 4 types.
- Example 2 of the original is not in the photos (only its last lines: "Clearly player 2 is a winner with more points. So, the answer is 2.").

How to think about it

- **Possible reading (inferred from Example 1):** score = number of boxes + (number of distinct sweet types if the number of distinct types is at least 4, otherwise 0). Print "Radha" if player 1 alone has the highest score, "Tie" if the highest score is shared, otherwise the index (starting from 1) of the single winner.

Complexity: O(total boxes). **Check:** Matches Example 1 only. The missing part of the rule could change the result for other inputs, so treat this as a reading of the statement, not a confirmed solution.

Solution (Python)

```
def contest(rows):                    # rows[i] = list of box numbers of player i+1
    score = []
    for r in rows:
        k = len(set(r))
        score.append(len(r) + (k if k >= 4 else 0))
    m = max(score)
    if score.count(m) > 1: return "Tie"
    i = score.index(m)
    return "Radha" if i == 0 else i + 1

# contest([[1,2,3,4,5,6],[3,3,3,4,4,4,5,5,5]]) -> "Radha"
```